Day 8

JavaScript Fundamentals

Notes, practice programs and solutions

Topics covered

- Introduction to JavaScript
- Variables: var, let and const
- Data types and operators
- Conditional statements and loops
- Functions
- Arrays and objects

Tasks

- Write programs using conditions and loops
- Create functions for basic calculations
- Perform array operations
- Create and access object properties
- Practice basic problem-solving programs

Expected output

- JavaScript practice programs
- Basic understanding of JavaScript fundamentals

How to run the code: save any example in a file such as `app.js` and run `node app.js`, or paste it into the browser console (F12).

1. Introduction to JavaScript

JavaScript is the programming language of the web. It runs in every browser to make pages interactive, and with Node.js it also runs on servers. HTML gives a page its structure, CSS gives it style, and JavaScript gives it behaviour.

```
console.log("Hello, JavaScript!");   // prints to the console
// This is a single-line comment
/* This is a
   multi-line comment */

Output:
Hello, JavaScript!
```

2. Variables: var, let and const

Keyword	Scope	Reassign?	Redeclare?	Use it?
var	Function	Yes	Yes	Avoid
let	Block	Yes	No	When value changes
const	Block	No	No	Default choice

```
let counter = 1;
counter = 2;              // OK
const PI = 3.14159;
// PI = 3;                // TypeError: assignment to constant variable

if (true) {
  var leaks = "var ignores block scope";
  let stays = "let stays inside the block";
}
console.log(leaks);       // works
// console.log(stays);    // ReferenceError
```

Rule of thumb: use `const` by default, switch to `let` only when the value must change, and avoid `var`.

3. Data Types

Primitives: string, number, boolean, undefined, null, bigint, symbol. **Non-primitives:** object (including arrays and functions).

```
console.log(typeof "Hello");     // string
console.log(typeof 42);          // number
console.log(typeof true);        // boolean
console.log(typeof undefined);   // undefined
console.log(typeof null);        // object  (a known JS quirk)
console.log(typeof [1, 2]);      // object
console.log(Array.isArray([1, 2]));  // true
```

4. Operators

Type	Operators	Example
Arithmetic	+ - * / % **	10 % 3 gives 1, 2 ** 3 gives 8
Comparison	== === != !== < > <= >=	5 === '5' is false
Logical	&& || !	a > 5 && b < 5

Type	Operators	Example
Assignment	= += -= *= /=	x += 3
Ternary	condition ? a : b	n % 2 === 0 ? 'even' : 'odd'
Nullish	??	null ?? 'default' gives 'default'

Important: always use === instead of ==. The loose version converts types silently, so `'5' == 5` is true.

5. Conditional Statements

Program: grade calculator (if / else if)

```
function gradeFor(marks) {
  if (marks >= 90) return "A";
  else if (marks >= 75) return "B";
  else if (marks >= 60) return "C";
  else if (marks >= 40) return "D";
  return "Fail";
}
console.log(gradeFor(82));
console.log(gradeFor(35));

Output:
B
Fail
```

Program: weekday or weekend (switch)

```
function dayType(day) {
  switch (day.toLowerCase()) {
    case "saturday":
    case "sunday":
      return "Weekend";
    default:
      return "Weekday";
  }
}
console.log(dayType("Sunday"));

Output:
Weekend
```

Program: leap year

```
const isLeapYear = (y) => (y % 4 === 0 && y % 100 !== 0) || y % 400 === 0;
console.log(isLeapYear(2024), isLeapYear(1900));

Output:
true false
```

6. Loops

for: multiplication table

```
const n = 5;
for (let i = 1; i <= 5; i++) {
  console.log(`${n} x ${i} = ${n * i}`);
}

Output:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

while: sum of digits

```
let num = 12345, sum = 0;
while (num > 0) {
  sum += num % 10;
  num = Math.floor(num / 10);
```

```
}
console.log(sum);

Output:
15
```

do...while, for...of and for...in

```
let tries = 0;
do { tries++; } while (tries < 3);        // runs at least once

for (const fruit of ["apple", "banana"]) console.log(fruit);   // values
for (const key in { name: "Asha", age: 21 }) console.log(key); // keys
```

Pattern and FizzBuzz

```
for (let row = 1; row <= 4; row++) console.log("*".repeat(row));

for (let i = 1; i <= 15; i++) {
  console.log(i % 15 === 0 ? "FizzBuzz" : i % 3 === 0 ? "Fizz" : i % 5 === 0 ? "Buzz" : i);
}

Output:
*
**
***
****
(then) 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

7. Functions

A function is a reusable block of code. JavaScript has three common styles:

```
function add(a, b) { return a + b; }                  // declaration
const subtract = function (a, b) { return a - b; };   // expression
const multiply = (a, b) => a * b;                     // arrow function
const divide = (a, b) => (b === 0 ? "Cannot divide by zero" : a / b);
const power = (base, exp = 2) => base ** exp;         // default parameter

console.log(add(8, 2), subtract(8, 2), multiply(8, 2));
console.log(divide(8, 0));
console.log(power(5), power(2, 10));

Output:
10 6 16
Cannot divide by zero
25 1024
```

More calculation functions

```
const areaOfCircle = (r) => +(Math.PI * r * r).toFixed(2);
const celsiusToFahrenheit = (c) => (c * 9) / 5 + 32;
const factorial = (n) => (n <= 1 ? 1 : n * factorial(n - 1));

const isPrime = (n) => {
  if (n < 2) return false;
  for (let i = 2; i * i <= n; i++) if (n % i === 0) return false;
  return true;
};

const fibonacci = (count) => {
  const seq = [0, 1];
  while (seq.length < count) seq.push(seq.at(-1) + seq.at(-2));
  return seq.slice(0, count);
};

console.log(areaOfCircle(7));        // 153.94
console.log(celsiusToFahrenheit(37)); // 98.6
console.log(factorial(5));            // 120
console.log(isPrime(29));             // true
console.log(fibonacci(8));            // [0, 1, 1, 2, 3, 5, 8, 13]
```

8. Arrays

```
const nums = [4, 8, 15, 16, 23, 42];

nums.push(50);     nums.pop();       // add / remove at end
nums.unshift(1);   nums.shift();     // add / remove at start

console.log(nums.map(v => v * 2));            // [8, 16, 30, 32, 46, 84]
console.log(nums.filter(v => v % 2 === 0));   // [4, 8, 16, 42]
console.log(nums.reduce((s, v) => s + v, 0)); // 108
console.log(nums.find(v => v > 15));          // 16
console.log(nums.includes(23), nums.indexOf(16)); // true 3
console.log(nums.slice(1, 3));                // [8, 15]
console.log([...nums].sort((a, b) => b - a)); // [42, 23, 16, 15, 8, 4]
console.log(Math.max(...nums), Math.min(...nums)); // 42 4
console.log([...new Set([1, 2, 2, 3, 3])]);   // [1, 2, 3]  (remove duplicates)
```

Method	What it does
map	Transforms every item and returns a new array

Method	What it does
filter	Keeps only the items that pass a test
reduce	Combines all items into a single value
find	Returns the first item that matches
sort((a, b) => a - b)	Sorts numbers (the default sort compares as text)

9. Objects

```
const student = {
  name: "Asha",
  age: 21,
  courses: ["HTML", "CSS", "JavaScript"],
  address: { city: "Aurangabad", state: "Maharashtra" },
  greet() { return `Hi, I'm ${this.name}`; },
};

console.log(student.name);              // dot access
console.log(student["age"]);            // bracket access
console.log(student.address.city);      // nested
console.log(student.courses[2]);        // array inside object
console.log(student.greet());           // method

student.email = "asha@example.com";     // add
student.age = 22;                       // update
delete student.email;                   // delete

console.log(Object.keys(student));
console.log("name" in student);         // true

const { name, address: { city } } = student;   // destructuring
```

Array of objects

```
const students = [
  { name: "Asha", marks: 88 },
  { name: "Ravi", marks: 67 },
  { name: "Neha", marks: 94 },
];
const topper = students.reduce((best, s) => (s.marks > best.marks ? s : best));
console.log(topper.name, topper.marks);      // Neha 94
console.log(students.map(s => s.name));      // ["Asha", "Ravi", "Neha"]
```

10. Problem-Solving Programs

```
const reverseString = (s) => s.split("").reverse().join("");
const isPalindrome = (s) => {
  const clean = s.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === reverseString(clean);
};
const countVowels = (s) => (s.match(/[aeiou]/gi) || []).length;

const wordFrequency = (text) => {
  const freq = {};
  for (const w of text.toLowerCase().split(/\s+/)) freq[w] = (freq[w] || 0) + 1;
  return freq;
};

const twoSum = (arr, target) => {
  const seen = {};
  for (let i = 0; i < arr.length; i++) {
    if (target - arr[i] in seen) return [seen[target - arr[i]], i];
    seen[arr[i]] = i;
  }
  return null;
};

console.log(reverseString("hello"));                 // olleh
console.log(isPalindrome("A man, a plan, a canal: Panama")); // true
console.log(countVowels("JavaScript"));              // 3
console.log(wordFrequency("the cat and the hat"));   // { the: 2, cat: 1, and: 1, hat: 1 }
```

```
console.log(twoSum([2, 7, 11, 15], 9));              // [0, 1]
```

11. Practice Challenges and Solutions

Challenge 1: print numbers from 1 to 100 divisible by 7

```
for (let i = 1; i <= 100; i++) {
  if (i % 7 === 0) console.log(i);
}

Output:
7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98
```

Challenge 2: write isEven(n) and filter an array with it

```
const isEven = (n) => n % 2 === 0;
console.log([3, 8, 12, 7, 20, 5].filter(isEven));

Output:
[ 8, 12, 20 ]
```

Challenge 3: second-largest number in an array

```
function secondLargest(arr) {
  const unique = [...new Set(arr)].sort((a, b) => b - a);
  return unique.length >= 2 ? unique[1] : null;
}
console.log(secondLargest([10, 45, 32, 45, 8]));

Output:
32
```

Challenge 4: count how many times each character appears

```
function charCount(str) {
  const counts = {};
  for (const ch of str) counts[ch] = (counts[ch] || 0) + 1;
  return counts;
}
console.log(charCount("banana"));

Output:
{ b: 1, a: 3, n: 2 }
```

Challenge 5: books with a price under 500

```
const books = [
  { title: "JavaScript Basics", price: 450 },
  { title: "Advanced CSS", price: 650 },
  { title: "HTML Handbook", price: 300 },
  { title: "React in Action", price: 799 },
  { title: "Git Essentials", price: 499 },
];
books.filter(b => b.price < 500)
     .forEach(b => console.log(`${b.title}: Rs. ${b.price}`));

Output:
JavaScript Basics: Rs. 450
HTML Handbook: Rs. 300
Git Essentials: Rs. 499
```

Day 8 Summary

- Use **const** by default, **let** when a value changes, and avoid **var**.
- Always compare with **===**.

- Use **if/else** and **switch** for decisions, and **for**, **while** and **for...of** for repetition.
- Functions package logic for reuse; arrow functions give a short syntax.
- **map**, **filter** and **reduce** are the most useful array methods.
- Objects store related data as key-value pairs, accessed with dot or bracket notation.